

Introduction to Digital Logic Design Lab

EECS 31L

Lab 3: Single-Cycle Processor

Ali Mortada
19849621

Februrary 22nd, 2025

1 Objective

The purpose of this lab was to design some of the modules needed in a RISC-V single-cycle processor. Fig. 1 shows the complete block diagram of the processor with all of its modules and internal wires. In this lab, only the flip flop (**FlipFlop**), instruction memory (**Instr_mem**), and register file (**RegFile**) modules were designed. The rest of the modules have already been implemented in previous labs or will be implemented in future labs. Fig. 2 lists the instructions that the processor handles.

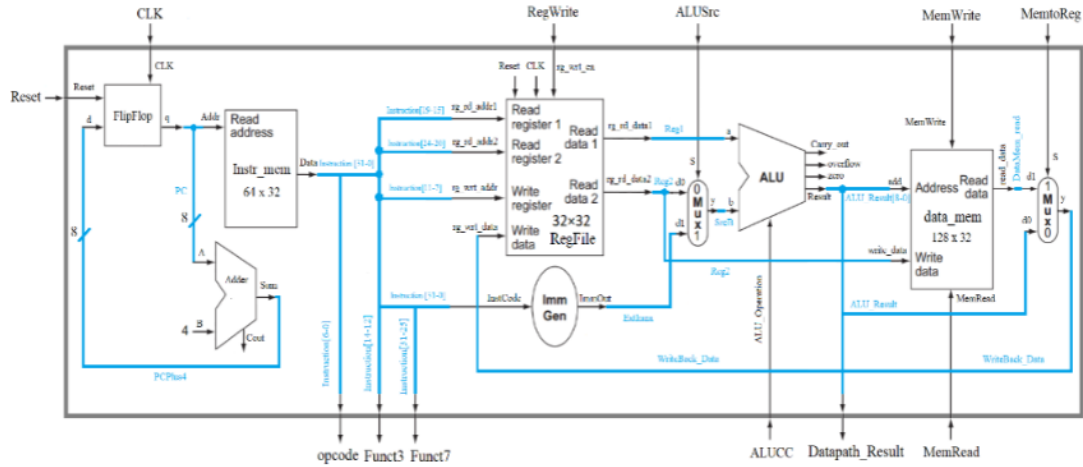


Figure 1: Block diagram for a RISC-V single-cycle processor.

imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	100	rd	0010011	NORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	100	rd	0110011	NOR
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

Figure 2: RISC-V instruction set.

2 Procedure

2.1 FlipFlop

The flip flop designed in this lab is a D-flip flop with a rising edge trigger and synchronous reset. It has three inputs: **d**, **reset**, and the clock signal **clk**, and one output, **q**. The input **d** and output **q** are 8 bits wide, while the rest of the signals are 1 bit wide. When the **reset** signal is high, the input **d** is blocked from the output **q** and the output is set to active low. When the **reset** signal is low, the output is set to match the input and updates at the rising edge of the clock signal. The timing diagram is shown in Fig. 3 and showcases the aforementioned behavior.

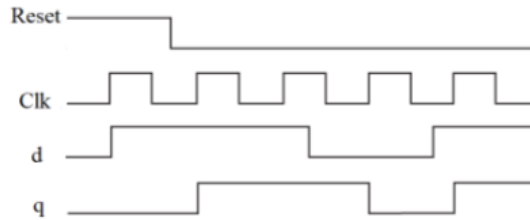


Figure 3: Timing diagram for the D-flip flop.

To implement this behavior in Verilog, sequential logic was used. Below is the implementation of the module behavior:

```
// define module behavior
always @(posedge clk) // trigger on rising edge
begin
    case (reset)
        1'b0: q <= d;
        1'b1: q <= 8'b0;
    endcase
end
```

The above code uses an always block that triggers on the rising edge of the clock signal to simulate the synchronous function of the module. Inside is a case statement based on the `reset` signal which, through non-blocking statements, allows the input to pass through depending on the signal's value.

2.2 Instruction Memory

The instruction memory module stores the instructions needed for the processor to function. Internally, the instructions are stored in an array that is 32 bits wide and 64 bits tall (meaning that it can store up to 64 instructions of 32-bit width). It has one input 8-bit input `addr` to prompt for the instruction's address in the memory and a 32-bit output `instruction` to give the instruction associated with the address. Fig. 4 shows the block diagram for the instruction memory module.

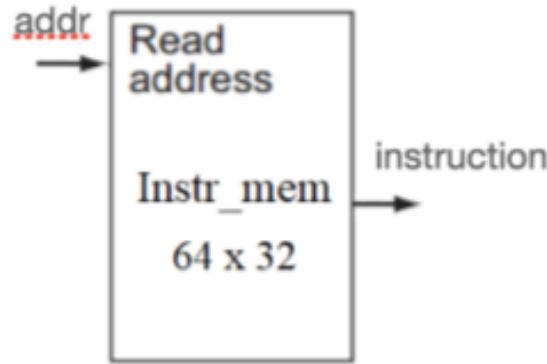


Figure 4: Block diagram for instruction memory.

Since this memory is byte-addressable, each instruction is split into four bytes. This means that to access the correct instruction using the address, the address must be divided by 4. This is implemented by shifting the address to the right by 2, which uses only the 6 most significant bits of the address when accessing the internal memory array. Below is the code used in the module:

```
// define instruction memory module behavior
reg [31:0] memory [63:0]; // 32 bits wide, 64 bits long
assign instruction = memory[addr >> 2];

initial
begin
    memory[0] = 32'h00007033; // and r0, r0, r0 32'h00000000
    memory[1] = 32'h00100093; // addi r1, r0, 1 32'h00000001
    memory[2] = 32'h00200113; // addi r2, r0, 2 32'h00000002
    memory[3] = 32'h00308193; // addi r3, r1, 3 32'h00000004
    memory[4] = 32'h00408213; // addi r4, r1, 4 32'h00000005
    memory[5] = 32'h00510293; // addi r5, r2, 5 32'h00000007
    memory[6] = 32'h00610313; // addi r6, r2, 6 32'h00000008
    memory[7] = 32'h00718393; // addi r7, r3, 7 32'h0000000B
    memory[8] = 32'h00208433; // add r8, r1, r2 32'h00000003
    memory[9] = 32'h404404b3; // sub r9, r8, r4 32'hfffffffe
    memory[10] = 32'h00317533; // and r10, r2, r3 32'h00000000
    memory[11] = 32'h0041e5b3; // or r11, r3, r4 32'h00000005
end
```

```

memory[12] = 32'h0041a633; // if r3 is less than r4 then r12 = 1 32'h00000001
memory[13] = 32'h007346b3; // nor r13, r6, r7 32'hfffffff4
memory[14] = 32'h4d34f713; // andi r14, r9, "4D3" 32'h000004D2
memory[15] = 32'h8d35e793; // ori r15, r11, "8d3" 32'hffffff8d7
memory[16] = 32'h4d26a813; // if r13 is less than 32'h000004D2 then r16 = 1 32'h00000000
memory[17] = 32'h4d244893; // nori r17, r8, "4D2" 32'hfffffb2C
end

```

2.3 Register File

Similar to the instruction memory, the register file also contains data that is stored in an internal array, this time of size 32 by 32 bits. However, the main difference is that the contents of this array can be read from as well as written to using the inputs. It features a reset, clock, and enable signal, two address read signals, two signals to read from those addresses, one signal to write an address, and one signal to write the data associated with that address. Each of the address signals are 5 bits wide, while the data bits are 32 bits wide. The register file can only be written to when the enable signal is high, on the rising edge of the clock. Unlike the flip flop, the reset is asynchronous, meaning that it sets all values in the internal array to zero regardless of whether the clock signal is high or low. Fig. 5 shows the block diagram for a register file.

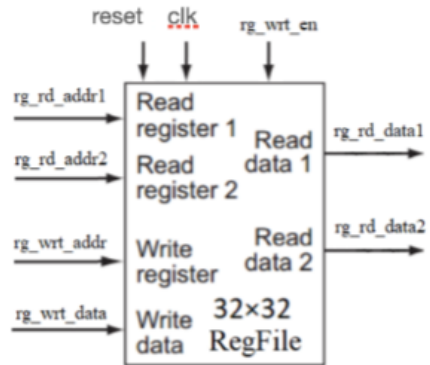


Figure 5: Block diagram for the register file.

The behavior of the module was defined using sequential logic. Below is the code:

```

// define register file module behavior
// internal storage for register file
reg [31:0] register_file [31:0]; // 32 bits wide, 32 bits long
integer i;

// reset (asynchronous)
always @(reset) begin
    if (reset) begin
        for (i = 0; i < 32; i = i + 1) begin
            register_file[i] = 32'h00000000;
        end
    end
end
end

```

```

// read from register file (asynchronous)
always @(rg_rd_addr1, rg_rd_addr2) begin
    rg_rd_data1 <= register_file[rg_rd_addr1];
    rg_rd_data2 <= register_file[rg_rd_addr2];
end

// write to register file (synchronous)
always @(posedge clk) begin
    if (reset == 1'b0 && rg_wrt_en == 1'b1) begin
        register_file[rg_wrt_addr] <= rg_wrt_data;
    end
end
end

```

The first always block defines the behavior of the reset signal by iterating through the array and setting each value in it to zero if the reset signal is high. The second always block uses non-blocking statements to output the members of the internal array based on the input addresses whenever they change. The third always block defines the behavior of writing to the register file on the rising edge of the clock whenever the enable signal is high and the reset signal is low.

3 Simulation Results

3.1 FlipFlop

To test the flip flop module, a test bench was written that does the following:

- Define a clock signal with a period of 20 ns
- Activate the reset signal for the first 100 ns
- After the first 100 ns, deactivate the reset signal and allow the input to pass through to the output

Below, in Fig. 6, is the simulation waveform:

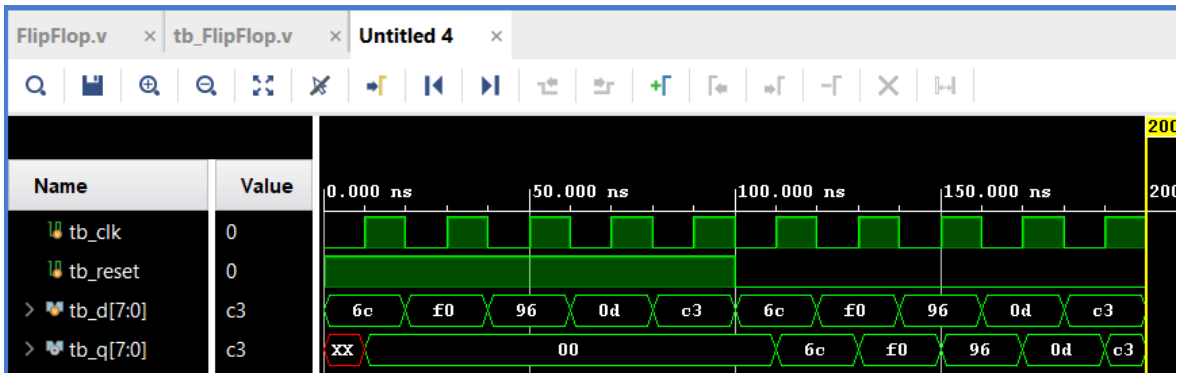


Figure 6: Simulation results for the D-flip flop.

In the first 100 ns, no matter what the input signal is, the output remains zero due to the reset signal being activated. After the reset signal is deactivated, the output is set to the input on the rising edge of each clock cycle. This shows that the code accurately describes the behavior of the flip flop.

3.2 Instruction Memory

To test the instruction memory module, another test bench was written to ensure that it follows the specification. The test bench specifies two cases, each run for 20 ns:

- `addr[7:2] = 16` (output should be `32'h4d26a813`)
- `addr[7:2] = 3` (output should be `32'h00308193`)

Below, in Fig. 7, is the simulation result:

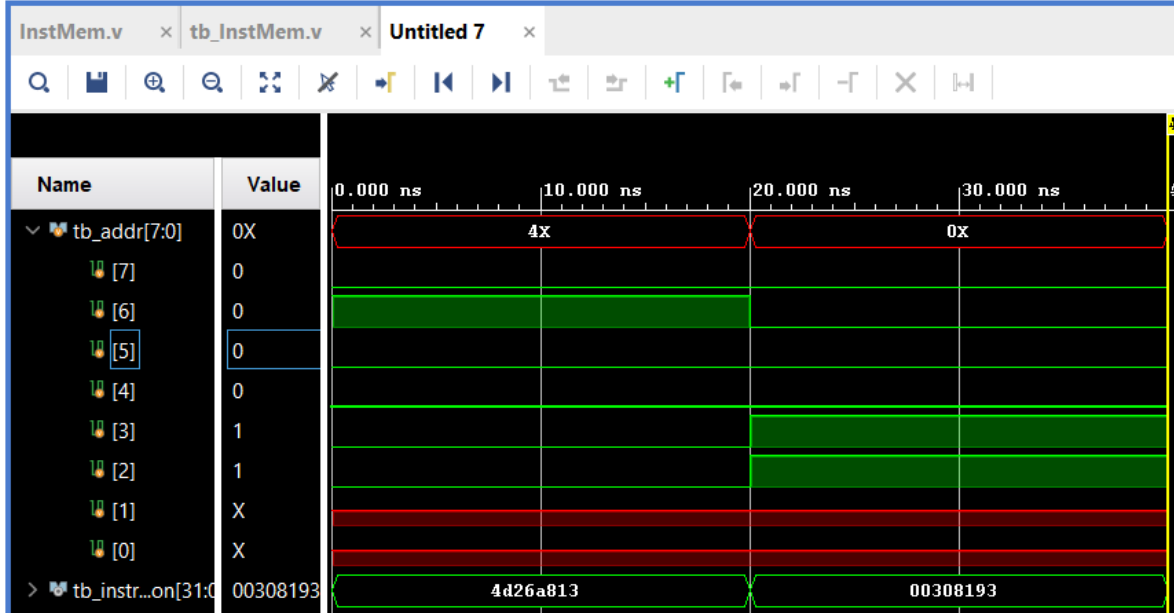


Figure 7: Simulation results for the instruction memory.

The simulation results show that the module produces the correct output for the given inputs. It is also important to note that since the address is shifted to the right by two bits to get the index for the instruction, it does not matter what the two least significant bits of the address are as the module will still give the correct output.

3.3 Register File

For the register file, the test bench written does the following:

- Set the clock signal with a period of 100 ns
- Activate the reset signal for the first 100 ns (one clock cycle), then deactivate it
- Activate the write enable signal
- Set the address to write into to the hexadecimal value `1e` (corresponding to register number 30)
- Set the value to write into the address to the hexadecimal value `ffffffff`

Below, in Fig. 8, are the simulation results:

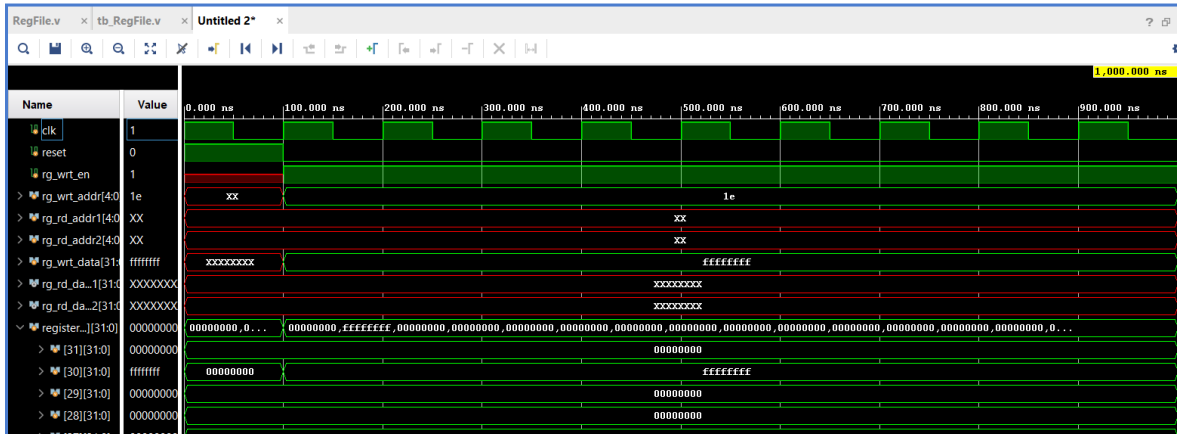


Figure 8: Simulation results for the register file.

When the reset signal is enabled and the write signal has not been enabled, the chosen address to write to has a value of zero. However, after the reset signal is deactivated and the write signal is enabled, the given data value (**ffffff**) is written to the correct address (register number 30). This means that the module accurately describes the behavior of a register file.